# Reporte de Agentic Loop

## 0. SKILLS CONFIGURADOS

arquitecto !’ DISEÑO-ARQUITECTURA (GLX-DISEÑO-ARQUITECTURA-V1)
tech_lead !’ MICROPLANIFICACION-DISEÑO (GLX-MICROPLANIFICACION-DISEÑO-V1)
desarrollador !’ FRONTEND-DESARROLLO (GLX-FRONTEND-DESARROLLO-V1)
qa !’ test (GLX-QA-VALIDACION-V1)

## 1. ANÁLISIS DE ARQUITECTURA

## Micro-ADR: Juego Arcade de Naves Espaciales (PROYEC-21)

### 1. CONTEXTO
Se requiere un juego web estilo arcade de los 80 (tipo Aero Fighter) completamente en HTML5, CSS y JS. El juego debe ejecutarse directamente en el navegador sin servidor backend. El requerimiento especifica explícitamente la estructura de carpetas con JS modular (game.js, controls.js, obstacles.js). Sin embargo, el entorno de ejecución es LOCAL (protocolo file:///), lo que impone restricciones críticas sobre el uso de módulos ES6 nativos.

### 2. DECISIÓN ARQUITECTÓNICA: Monolito de Archivo Único con Namespaces JS (IIFE Pattern)
**Entorno detectado: LOCAL / file:/// — SIN SERVIDOR HTTP**

Dado que el requerimiento indica HTML5/CSS/JS puro sin mencionar servidor, y la arquitectura propuesta usa múltiples archivos JS, se aplica la regla de evaluación de entorno obligatoria:

- **PROHIBIDO**: `<script type='module'>`, `import/export` ES6 nativos entre archivos separados (fallarán con CORS policy en file:///)
- **DECISIÓN**: Todo el JavaScript se consolida en un único archivo `js/game.js` usando el patrón de Namespaces con IIFEs (Immediately Invoked Function Expressions) y objetos globales para simular la modularidad requerida por el cliente sin romper la compatibilidad local.
- **Estructura interna de game.js**: Módulos lógicos separados por comentarios de sección — GameEngine, Controls, ObstacleManager, PowerUpManager, ScoreSystem, Renderer — todos dentro de un único scope global controlado.
- **Canvas API**: Se usará HTML5 Canvas 2D como motor de renderizado. Es la tecnología más madura, sin dependencias externas, ideal para juegos arcade 2D. NO se usará WebGL (YAGNI: la estética pixel-art de los 80 no lo requiere).
- **Audio**: Web Audio API nativa del navegador. NO se usará Howler.js ni ninguna librería externa (YAGNI).
- **Gráficos**: Dibujados programáticamente con Canvas 2D API usando formas geométricas y pixel-art sintético. NO se referencian archivos de imagen binarios (.png/.webp). Se usarán funciones de dibujo puras para la nave, obstáculos y efectos, garantizando que el juego funcione sin assets externos.

### 3. JUSTIFICACIÓN BASADA EN YAGNI
- **¿Por qué NO microservicios/backend?** El juego es 100% client-side. No hay persistencia de datos entre sesiones requerida en el MVP. El scoreboard es en memoria durante la sesión.
- **¿Por qué NO múltiples archivos JS con módulos?** El entorno file:/// bloquea CORS en imports nativos. Un solo archivo JS elimina este riesgo sin sacrificar organización interna.
- **¿Por qué NO Three.js o Phaser.js?** KISS + YAGNI. Canvas 2D nativo es suficiente para estética arcade 80s. Agregar un framework de juegos añade 1-2MB de dependencia externa y curva de aprendizaje innecesaria para este scope.
- **¿Por qué NO localStorage para scores?** El MVP no lo requiere explícitamente. Se puede agregar en iteración posterior con 3 líneas de código.
- **¿Por qué NO responsive.css separado?** El juego usa Canvas con dimensiones fijas (estética arcade). Un media query en styles.css es suficiente para centrar el canvas en mobile.

### 4. RIESGOS ASUMIDOS POR SIMPLIFICAR

- **Riesgo 1**: Si en el futuro se requiere un servidor HTTP (para leaderboards globales), la refactorización a módulos ES6 requerirá separar el game.js en múltiples archivos. Mitigación: la estructura interna por namespaces facilita esta extracción.
- **Riesgo 2**: Sin assets de audio externos, los efectos de sonido serán generados con oscillators de Web Audio API (estilo chiptune). Esto es coherente con la estética arcade 80s pero puede percibirse como limitado.
- **Riesgo 3**: El rendimiento del Canvas 2D puede degradarse en dispositivos muy antiguos con muchos obstáculos simultáneos. Mitigación: object pooling implementado en ObstacleManager para reutilizar objetos.

### 5. STACK TECNOLÓGICO FINAL
- **Renderizado**: HTML5 Canvas 2D API (nativo)
- **Lógica**: JavaScript ES5/ES6 compatible (sin módulos), patrón IIFE + Namespace
- **Estilos**: CSS3 puro (sin frameworks)
- **Audio**: Web Audio API (nativo, síntesis chiptune)
- **Dependencias externas**: CERO
- **Servidor requerido**: NINGUNO

### 6. COMPONENTES DEL JUEGO
- **Nave del jugador**: Movimiento con flechas (!•!’ giro, !‘!“ acelerar/frenar con velocidad mínima >0)
- **5 tipos de obstáculos**: Cometa, Asteroide, Planeta, OVNI (enemigo con IA simple), Nebulosa (obstáculo de área)
- **3 tipos de power-ups**: Escudo temporal, Turbo de velocidad, Disparo múltiple (triple shot)
- **3 niveles de dificultad**: Progresiva automática por tiempo de supervivencia
- **Sistema de puntuación**: Tiempo de supervivencia + objetos destruidos + power-ups recolectados
- **Estética**: Pixel-art sintético, paleta de colores neón sobre fondo negro, fuente monospace

## 2. MÉTRICAS DE ITERACIÓN (QA)

- index.html | Estado: APROBADO | Iteraciones: 2
- js/game.js | Estado: APROBADO | Iteraciones: 1
- css/styles.css | Estado: APROBADO | Iteraciones: 1
- README.md | Estado: SKIPPED (doc) | Iteraciones: 0

## 3. MÉTRICAS DE TOKENS IA

Tokens de entrada (input): 139,083
Tokens de salida (output): 37,600
Total tokens: 176,683
Modelo utilizado: claude-sonnet-4-6

### Ø=Üh  Ø=Ü» ~17.6 horas-hombre ahorradas
"H squad de 4 personas × 2.2 días

Desglose por Agente:

DISEÑO-ARQUITECTURA: !“2,543 input | !‘1,757 output | 1 llamadas | ~1.8h ahorradas
MICROPLANIFICACION-DISEÑO: !“20,079 input | !‘9,499 output | 4 llamadas | ~6h ahorradas
FRONTEND-DESARROLLO: !“46,265 input | !‘20,219 output | 5 llamadas | ~5.4h ahorradas
test: !“70,196 input | !‘6,125 output | 5 llamadas | ~4.4h ahorradas